Learning Topology-Aware Cooperative Caching Policies for IoT Edge Networks

Abdullah Al Mujahid, Asir Abrar, Monideepa Kundu Sinthi

April 02, 2026

1 Introduction

Edge caching has emerged as a fundamental mechanism for reducing latency, alleviating backhaul congestion, and enabling real-time services in distributed systems. With the rise of edge computing and IoT, bringing content closer to users has become essential for supporting latency-sensitive applications such as autonomous driving, augmented reality, and real-time analytics [1–3]. Early works on distributed caching and content delivery demonstrated the benefits of placing content at the network edge to reduce access delays and improve scalability [4, 5].

However, most traditional caching strategies assume static network conditions and rely heavily on global popularity distributions. In practice, real-world edge networks exhibit dynamic behavior due to mobility, fluctuating workloads, and changing connectivity patterns. Even when the underlying infrastructure (e.g., WiFi access points) remains fixed, effective connectivity evolves over time due to user movement and access patterns. This mismatch between static assumptions and dynamic environments leads to suboptimal caching decisions and degraded performance.

Recent advances have explored learning-based and cooperative caching strategies to address these challenges. Deep reinforcement learning (DRL) has been applied to adapt caching decisions in dynamic settings [6, 7], while multi-agent approaches enable coordination among distributed nodes [8]. In parallel, emerging application domains such as vehicular networks, UAV-assisted systems, and satellite-edge integration highlight the importance of mobility-aware and cooperative caching [9, 10]. These works demonstrate that caching decisions must account for both network structure and temporal variability.

Despite these advances, two critical gaps remain. First, many approaches assume either predictable mobility models or require strong coordination mechanisms, limiting their applicability in real-world deployments. Second, most formulations do not explicitly incorporate the cost of adapting cache placements under topology changes, leading to unstable or inefficient policies in dynamic environments.

In this work, we formulate edge caching as a topology-aware cooperative optimization problem over a graph-based network. The system is modeled as a static base topology derived from real-world traces, with stochastic perturbations capturing variability in connectivity. The objective is to minimize the expected content access cost while jointly considering storage constraints, redundancy penalties, and cache relocation overhead.

To solve this problem, we propose a hybrid framework combining greedy optimization and deep reinforcement learning. The greedy component provides an efficient initial placement based on local utility, while the DRL component learns adaptive policies that respond to topology perturbations and dynamically relocate content. This combination enables both scalability and robustness under uncertainty.

Our contributions are threefold: (i) a unified problem formulation that integrates topology-awareness, uncertainty, and relocation cost, (ii) a hybrid optimization approach combining heuristic and learning-based methods, and (iii) a trace-driven evaluation demonstrating robustness under dynamic conditions.

2 Challenges of IoT Edge Caching

Edge caching faces challenges due to changing demand, user mobility, limited storage and computing resources, and evolving relationships between nodes. Demand varies across locations and over time, and user movement creates correlated and non-stationary request patterns that do not match classical caching assumptions. Limited edge resources and increasing network density also make coordination and scalability more difficult. These factors make caching at the IoT edge more complex than in traditional systems and require strategies that consider mobility and network structure.

IoT workloads exhibit strong spatial and temporal variability, where data popularity differs across locations and changes over time due to contextual or event driven factors [1, 2]. User mobility further creates correlation across edge nodes, as content requested at one access point may soon be requested at neighboring nodes when users move [11]. At the same time, the arrival and departure of users can cause sudden changes in local popularity distributions. These effects challenge classical caching models, which typically assume stable, independent, and stationary request streams [12]

Edge nodes such as IoT gateways or access points operate under constrained storage budgets. Unlike centralized cloud servers or CDNs, edge devices must carefully allocate limited cache capacity among competing content. This creates a trade-off between replicating content across nearby nodes to increase availability and diversifying cached content to reduce redundancy. In addition to storage constraints, IoT edge devices often have limited computational power, energy resources, and communication bandwidth which limit the feasibility of fully centralized optimization and motivate scalable, distributed approaches. Furthermore, although the physical infrastructure of edge networks may remain fixed, the logical interaction structure among nodes evolves over time due to user mobility and varying connectivity condition. So, caching strategies must therefore adapt not only to changing demand but also to evolving inter-node relationships. Also as future 5G/6G and IoT deployments become increasingly dense, the number of potential cooperative relationships grows, making coordination and optimization more complex and less practical. Any effective caching strategy must therefore scale efficiently with network size while maintaining decentralized operation.

Collectively, these challenges motivate IoT edge caching strategies that consider mobility-induced relationships and adapt to dynamic demand under limited resources. Effective solutions must consider how nodes interact over time and coordinate caching decisions in a scalable manner.

3 Related Work

Edge caching has been widely investigated across wireless networks, mobile systems, and IoT-enabled edge computing. Existing work can be broadly categorized into (i) optimization-based cooperative caching, (ii) mobility-aware and social-aware caching, (iii) reinforcement learning–based caching, and (iv) knowledge and graph-based caching approaches. Despite significant progress, current approaches still exhibit limitations in explicitly modeling dynamic, mobility-derived topology as a first-class learning object.

3.1 Optimization-Based Cooperative Caching

Several works have done in cooperative caching as an optimization problem under storage and bandwidth constraints. Blasco and Gunduz [7] proposed a learning-based proactive caching framework that models content popularity dynamics using online learning and solves cache placement via convex optimization. Similarly, Maddah-Ali and Niesen [13] introduced coded caching to improve network efficiency through coordinated placement and delivery strategies. These approaches demonstrate that coordination among nodes can significantly improve system performance. However, they typically assume static or centrally known network topology and do not adaptively learn the mobility-induced relationships between nodes. Xu et al. proposed a cooperative caching framework for UAV-assisted vehicular networks that combines LSTM-attention-based popularity prediction with coalition-game-based cache optimization, making it most closely related to optimization-based cooperative caching [3]. Ma et al. proposed a delay-oriented caching strategy for LEO satellite-assisted edge caching networks by modeling a content request popularity with a Zipf distribution and average transmission delay using stochastic geometry, and by optimizing satellite caching probabilities through a particle swarm optimization based minimum-delay strategy, which improves delay performance through probabilistic cache allocation, but it does not explicitly represent mobility-driven relationships among edge nodes or use topology-aware graph learning for cooperative caching decisions [10].

3.2 Mobility-Aware and Social-Aware Caching

User mobility has been recognized as an important factor in caching efficiency. Wang et al. [14] explored mobility prediction to improve caching decisions in mobile edge networks. Mardham et al. [15] proposed opportunistic distributed caching in delay-tolerant networks using mobility patterns. Cabaniss and Madria [16] propose a socially-aware caching strategy for DTNs that leverages contact frequency and social relationships to guide content placement. Their approach improves delivery probability by exploiting community structures but relies on predefined social metrics rather than explicit topology modeling. While these works acknowledge the correlations, cooperation is often driven by predefined mobility predictions or social heuristics rather than by learning the evolving topological structure among edge nodes.

3.3 Reinforcement Learning-Based Edge Caching

Reinforcement learning (RL) has emerged as a promising technique for dynamic caching under non-stationary demand. Wang et al. [17] proposed a multi-agent deep reinforcement learning framework MARL with bandwidth-efficient inter-agent communication to improve cooperative edge caching performance under dynamic demand. Although it introduces cooperative behavior, the underlying topology is typically implicit. Agents exchange limited information or coordinate through reward shaping, but the structural graph of inter-node relationships is not explicitly modeled or learned as a dynamic entity. Jiang et al. proposed cooperative content caching in vehicular edge computing networks as a Markov decision process and proposed a two-stage deep reinforcement learning framework for joint caching decisions across vehicles and roadside units to reduce long-term delay and improve cache hit rate under dynamic vehicular mobility, heterogeneous node constraints, and a large caching policy space [18]. Dhara et al.

proposed a caching framework for post-disaster Space-Air-Ground Integrated Networks (SAGIN) that uses a Contextual Multi-Armed Bandit and a Federated MAB model to support real-time content retrieval across satellites, UAVs, base stations, and vehicles. The framework addresses dynamic caching under heterogeneous network conditions, limited storage, mobility, and changing request patterns by using contextual features such as content popularity, freshness, and cache-size ratio into caching decisions, while the federated extension improves adaptation by sharing learned reward information among distributed nodes [19]. Cao et al. proposed a mobility-aware cooperative caching framework for vehicular edge computing based on federated distillation and deep reinforcement learning which predicts popular content using mobility information and privacy-aware distributed learning, and then uses reinforcement learning to decide how that content should be cached across neighboring roadside units [20]. Zhao et al. [21] proposed a collaborative caching and offloading method for vehicular edge computing that predicts the stability of links between vehicles, analyzes the popularity of historical offloading tasks for caching, and then uses deep reinforcement learning to make partial offloading decisions.

3.4 Knowledge and Graph Based Caching Approaches

More recently, structured representations such as knowledge graphs have been introduced to enhance caching decisions. Wang et al. [22] proposed a knowledge graph based reinforcement learning framework for cooperative caching in mobile edge computing systems. Their approach captures semantic relationships among contents and network states.

Despite incorporating structured representations, most existing work focuses on content level relationships rather than explicitly modeling time varying inter node mobility graphs. The integration of graph learning methods to directly encode mobility-induced topology in cooperative caching remains largely unexplored.

4 Limitations of Current State-of-the-Art

Despite significant advances in edge and cooperative caching, existing state-of-the-art (SOTA) approaches exhibit structural limitations that reduce their effectiveness in dynamic IoT edge environments.

4.1 Node-Centric Optimization

A large body of caching research focuses on node-local decision making, where each edge node independently optimizes its cache based on observed local demand. Classical cache replacement strategies [12] and learning-based small-cell caching methods [7] primarily rely on per-node optimization. While such approaches improve scalability, they neglect mobility induced inter-node demand correlations.

Even proactive caching frameworks for 5G systems [23] and learning-based edge caching strategies [6] often treat nodes as independent optimization units. As a result, independently optimized caches may replicate identical content unnecessarily or fail to anticipate demand migration across neighboring nodes.

4.2 Static Cooperation Assumptions

Cooperative caching has been extensively studied in wireless and distributed systems. Foundational works such as FemtoCaching [4] and distributed wireless caching [5] demonstrate the benefits of coordination among helpers or small cells. However, these approaches typically assume static helper relationships or fixed deployment structures.

Similarly, opportunistic and delay-tolerant network caching frameworks [15, 16, 24] rely on predefined collaboration patterns or social-context assumptions. In highly dynamic IoT edge environments, where user mobility continuously reshapes inter-node interactions, fixed cooperation structures may not accurately reflect evolving demand relationships.

4.3 Implicit Topology Modeling in Learning-Based Methods

Recent machine learning and multi-agent reinforcement learning (MARL) approaches improve adaptability under dynamic demand [8, 17, 25]. However, topology is often incorporated implicitly through shared rewards, heuristic communication, or limited information exchange rather than being explicitly modeled as a structured graph.

Without formal graph based modeling of node interactions, these approaches cannot fully exploit mobility induced structural correlations. Although graph neural networks (GNNs) have shown strong potential for modeling relational dependencies in communication networks [26, 27], their systematic integration into cooperative caching remains limited.

4.4 Limited Use of Real Mobility Data

Many evaluation studies rely on synthetic mobility models or simplified traffic assumptions [14, 25]. While analytically convenient, such settings do not capture realistic user association patterns and handoff dynamics observed in operational networks.

Public mobility traces, such as the Dartmouth campus dataset [11], reveal complex and nonuniform interaction structures that are often overlooked in synthetic evaluations. The limited use of real mobility data restricts the external validity of many proposed caching strategies.

4.5 Scalability and Generalization Challenges

Centralized optimization based methods, including information theoretic caching formulations [13], may achieve strong theoretical guarantees but become computationally expensive in large scale deployments. Conversely, fully decentralized heuristics may scale efficiently yet provide suboptimal coordination.

Additionally, policies trained under specific demand or topology assumptions may fail to generalize across heterogeneous environments, including fog and IoT architectures [1, 2]. The absence of explicit structural modeling further limits robustness across varying network densities and mobility regimes.

Overall, current SOTA approaches are constrained by node centric optimization, static cooperation assumptions, implicit topology handling, limited mobility grounded evaluation, and scalability generalization trade offs. These limitations motivate the need for a topology aware cooperative caching framework that explicitly models mobility derived interaction graphs and learns coordinated caching policies suited for dynamic IoT edge systems.

5 Problem Statement

We consider a distributed edge caching system in which a set of edge nodes collaboratively serve content requests generated by mobile users. Each node has limited cache capacity and may satisfy a request either locally, by retrieving the content from another node in the network, or by accessing the origin server. Since users move across nodes, the system exhibits mobility-induced correlations: nodes that frequently exchange users tend to observe related access patterns and may benefit from coordinated caching decisions.

We represent the system by a weighted graph $G = (V, E, W)$, where V is the set of edge nodes, E is the set of connectivity relations, and $W = \{w_{ij}\}$ captures mobility-induced interaction strength between nodes. Larger values of w_{ij} indicate stronger relationships, such as frequent handoffs or persistent overlap in user movement. Unlike classical caching systems in which each node is optimized independently, this setting gives rise to a coupled decision problem: content placement at one node affects accessibility, redundancy, and traffic at other nodes.

In practice, the network topology is not perfectly stable. Nodes may be added or removed due to mobility, intermittent availability, failures, or deployment changes. To model this uncertainty, we consider a perturbed graph $\tilde{G} = (\tilde{V}, \tilde{E}, \tilde{W})$ sampled from a perturbation process $\mathcal{P}(G)$, which captures random node arrivals and departures. A useful caching policy must therefore remain effective not only for the nominal graph G, but also on average under such perturbations.

5.1 Subproblems

5.1.1 Demand Modeling under Partial Observability

The mobility trace provides user–node association information, but it does not include content-level request logs. As a result, demand must be synthesized while preserving realistic characteristics such as skewed popularity, spatial locality, and temporal variability. This creates a demand modeling problem: the request process should be realistic enough to support meaningful caching evaluation, yet controlled enough to avoid introducing artificial bias.

5.1.2 Topology-Aware Cooperative Placement

Caching decisions are coupled through the graph structure. If each node optimizes only for its own local demand, neighboring nodes may store the same content repeatedly, wasting cache space. On the other hand, fully centralized global optimization is costly and often unrealistic in distributed systems. The challenge is to exploit topology-aware cooperation without requiring complete global synchronization.

5.1.3 Access–Replication Tradeoff

Replicating content across more nodes improves accessibility and lowers retrieval latency, but it also consumes storage and incurs maintenance overhead. A good caching strategy must therefore balance the gain in access efficiency against the cost of storing and maintaining replicas.

5.1.4 Relocation under Topology Perturbation

When the topology changes due to node additions or removals, a previously efficient cache placement may become suboptimal. The system should be able to relocate cached content to adapt to perturbations, but relocation itself incurs communication and replacement cost. Thus, the policy must balance robustness against instability.

5.2 Formal Problem Formulation

Let C denote the content catalog. For each node $i \in \tilde{V}$ and content item $c \in C$, define the binary placement variable

$$x_{i,c} = \begin{cases} 1, & \text{if content } c \text{ is cached at node } i, \\ 0, & \text{otherwise.} \end{cases}$$

Let $x^0_{i,c}$ denote the initial cache state before perturbation. Let $P_{i,c}$ denote the probability that node i requests content c, and let f_c denote the replication-related cost of content c, which may reflect update frequency, storage burden, or a combined maintenance factor.

Let $d(i, j)$ be the shortest-path distance between nodes i and j in the perturbed graph $\tilde{G}$, and let $L(i, j)$ denote the latency of retrieving content from node j to node i. We assume that $L(i, j)$ is a non-decreasing function of graph distance. Let L_{local} denote the latency of serving content from the local cache, and let L_{origin} denote the latency of retrieving content from the origin server.

To simplify notation, define the best non-local retrieval latency for content c at node i under placement x and perturbed topology $\tilde{G}$ as

$$A_{i,c}(x, \tilde{G}) = \min\left(L_{\text{origin}}, \min_{\substack{j \in \tilde{V} \\ j \neq i \\ x_{j,c}=1}} L(i, j) \right).$$

If no other node stores content c, then $A_{i,c}(x, \tilde{G}) = L_{\text{origin}}$.

The expected access cost is

$$C_{\text{access}}(x, \tilde{G}) = \sum_{i \in \tilde{V}} \sum_{c \in C} P_{i,c} \left[x_{i,c} L_{\text{local}} + (1 - x_{i,c}) A_{i,c}(x, \tilde{G}) \right].$$

The replication cost is

$$C_{\text{rep}}(x, \tilde{G}) = \sum_{i \in \tilde{V}} \sum_{c \in C} \gamma f_c x_{i,c},$$

where $\gamma > 0$ scales the importance of replication overhead.

To discourage redundant placement among strongly related nodes, we define the topology-aware redundancy penalty

$$C_{\text{red}}(x, \tilde{G}) = \sum_{c \in C} \sum_{\substack{i,j \in \tilde{V} \\ i<j}} \tilde{w}_{ij}\, x_{i,c} x_{j,c}.$$

To capture the cost of adapting the cache after perturbation, we define the relocation cost

$$C_{\text{rel}}(x, x^0) = \sum_{i \in \tilde{V}} \sum_{c \in C} \delta_c \, |x_{i,c} - x^0_{i,c}|,$$

where δ_c denotes the cost of adding or removing content c at a node.

The topology-aware cooperative caching problem is therefore formulated as the following stochastic optimization problem:

$$\min_x \; \mathbb{E}_{\tilde{G} \sim \mathcal{P}(G)} \left[C_{\text{access}}(x, \tilde{G}) + \lambda C_{\text{rep}}(x, \tilde{G}) + \beta C_{\text{red}}(x, \tilde{G}) + \mu C_{\text{rel}}(x, x^0) \right],$$

where $\lambda > 0$, $\beta > 0$, and $\mu > 0$ control the tradeoff between access efficiency, replication overhead, redundancy reduction, and relocation stability.

This objective captures the central design challenge of the system. Aggressive replication can reduce latency but increases replication and redundancy cost. Conservative placement may reduce overhead but increase expensive remote retrievals. Frequent relocation may adapt well to perturbations but can incur substantial update cost. The desired policy is therefore one that minimizes expected overall cost under random topology perturbations.

5.3 Constraints

The optimization problem is subject to the following practical constraints.

Cache Capacity Constraint. Each node has limited storage and therefore cannot cache the entire catalog. If K_i denotes the cache capacity of node i, then the total size or number of cached items must remain within its budget:

$$\sum_{c \in C} x_{i,c} \le K_i, \qquad \forall i \in \tilde{V}.$$

This ensures that the solution is feasible under node-level storage limitations.

Binary Placement Constraint. Caching is a discrete decision: an item is either stored at a node or it is not. Therefore,

$$x_{i,c} \in \{0, 1\}, \qquad \forall i \in \tilde{V},\ \forall c \in C.$$

This makes the placement problem combinatorial.

Latency Ordering Constraint. The system assumes a natural service hierarchy: local cache access is cheapest, inter-node retrieval is more expensive, and origin retrieval is most expensive:

$$L_{\text{local}} < L(i, j) \leq L_{\text{origin}}, \qquad \forall i, j \in \tilde{V},\ i \neq j.$$

This reflects the fact that cooperation is beneficial but not free.

Perturbation Feasibility Constraint. Cache placement after perturbation is only defined on nodes that remain present in $\tilde{V}$. Newly added nodes may begin with empty cache state, while removed nodes cannot participate in storage or retrieval. This ensures that placement and relocation decisions are consistent with the realized perturbed topology.

Topology-Dependent Communication Constraint. Inter-node retrieval cost depends on the perturbed graph structure through $L(i, j)$. Thus, cooperation is not assumed to be globally uniform or free; it is mediated by graph connectivity and path cost. This ensures that the topology plays a direct and meaningful role in the optimization problem.

6 Dataset

To evaluate topology aware cooperative caching under realistic mobility conditions, we use the CRAWDAD Dartmouth Campus Wi-Fi dataset [11]. This dataset was collected at Dartmouth College between April 2001 and June 2004 and contains large scale wireless network traces covering several thousand users and over 450 access points (APs) deployed across campus buildings and includes syslog events, SNMP polling logs, packet level tcpdump traces, and derived mobility records. For this study, we will primarily use the movement traceset,which shows the order in which each wireless device connects to different access points over time on campus.All data has been anonymized to protect user privacy, and MAC/IP addresses are consistently sanitized across traces.

The movement dataset contains time ordered records of which access point each wireless device was connected to over time. Each device has a trace file identified by its anonymized MAC address. The timestamp is recorded in Unix time (seconds since January 1, 1970). A special state called OFF indicates that the device is not connected to the wireless network.

In addition to the movement traces, we use the file APlocations.csv, which provides spatial information for most access points. Each entry contains: AP name, x coordinate, y coordinate, z coordinate (floor level) This spatial data allows optional analysis of physical proximity between APs and comparison with mobility based connectivity patterns.

The movement dataset does not include actual content request information. Therefore, we assume a global content catalog and simulate user requests following a Zipf distribution, which reflects realistic popularity observation in content delivery system. As the device moves between APs, its demand pattern follows its mobility path, creating a relaistic demand patterns and across the network and allows us to

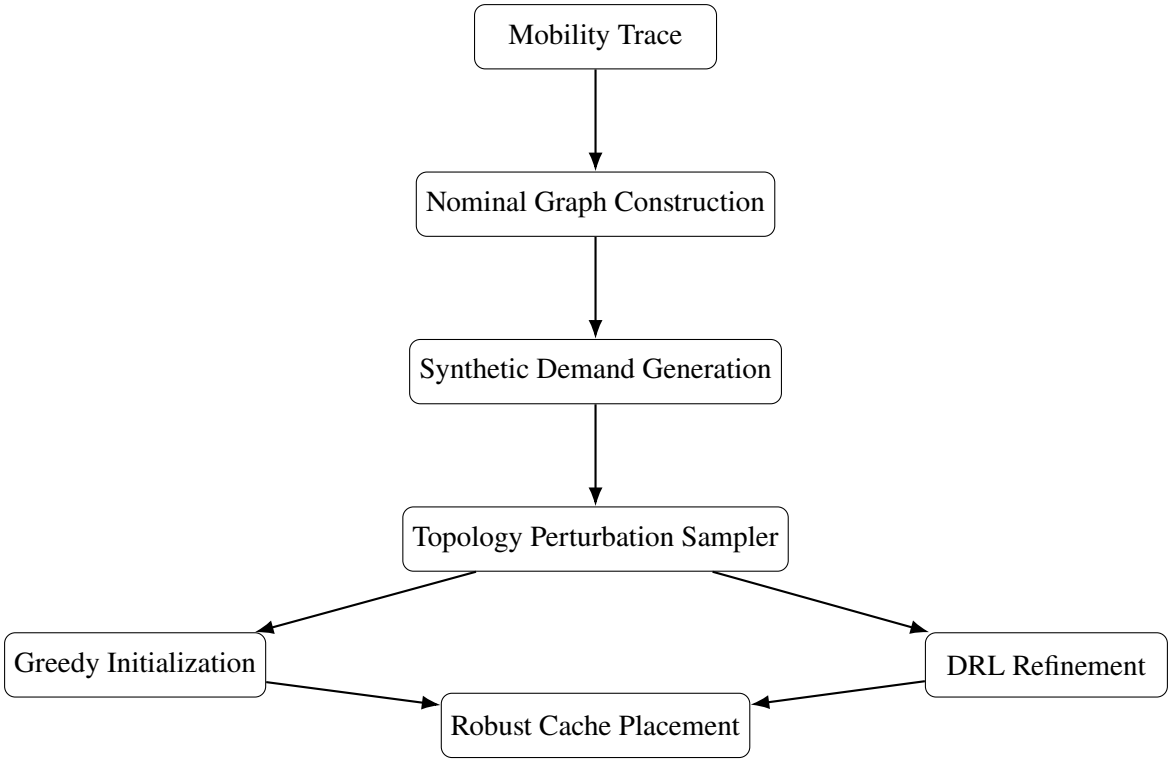

Figure 1: Methodology pipeline: the system builds a nominal mobility graph, generates demand and perturbations, computes a greedy initial placement, and then refines it through DRL.

evaluate cooperative caching strategies even though the original dataset does not contain real content request logs.

7 Methodology

7.1 Overview

We propose a two-stage topology-aware cooperative caching framework that combines a greedy initialization procedure with a deep reinforcement learning (DRL) refinement phase. The greedy stage provides a fast, interpretable, and cost-aware initial placement, while the DRL stage refines this placement under random topology perturbations. This hybrid design is motivated by two considerations. First, the caching problem is combinatorial and difficult to solve exactly at scale. Second, directly training a DRL agent over the full placement space is inefficient without a strong initialization.

The complete pipeline consists of the following steps:

1. Construct a mobility-derived nominal graph $G = (V, E, W)$ from the dataset.
2. Generate synthetic request probabilities $P_{i,c}$ over the content catalog.
3. Create random perturbed graph instances $\tilde{G} \sim \mathcal{P}(G)$ through node addition and removal.
4. Compute an initial placement using a topology-aware greedy algorithm.
5. Refine the placement using DRL, with reward defined by the negative expected total cost.

7.2 Graph Construction and Perturbation Model

We first construct a nominal weighted graph $G = (V, E, W)$ from user handoff traces. For each pair of nodes $i, j \in V$, let H_{ij} denote the number of observed handoffs from i to j. We define the normalized interaction weight as

$$w_{ij} = \frac{H_{ij}}{\sum_k H_{ik}}.$$

Edges with negligible transition frequency may be removed by thresholding.

To model topology uncertainty, we define a perturbation process $\mathcal{P}(G)$ that generates a perturbed graph $\tilde{G} = (\tilde{V}, \tilde{E}, \tilde{W})$ by randomly removing and adding nodes. When a node is removed, all incident edges are deleted. When a node is added, its incident edges are inferred from the mobility model or sampled from historical transition statistics. This perturbation model is used both during greedy evaluation and DRL training.

7.3 Synthetic Demand Model

Since the dataset does not include content requests, demand is generated synthetically. Let C be the content catalog. Global content popularity follows a Zipf distribution:

$$P(c) = \frac{c^{-\alpha}}{\sum_{m=1}^{|C|} m^{-\alpha}},$$

where $\alpha > 0$ controls the skewness.

To capture spatial locality, the node-specific demand distribution is defined as

$$P_{i,c} = \frac{\eta_{i,c} P(c)}{\sum_{c' \in C} \eta_{i,c'} P(c')},$$

where $\eta_{i,c}$ is a locality bias factor for node i and content c. This allows different nodes to prefer different subsets of the catalog.

7.4 Cost Components

The methodology uses the same cost terms as the problem formulation.

For a placement x on a perturbed graph $\tilde{G}$, define the best non-local retrieval latency as

$$A_{i,c}(x, \tilde{G}) = \min\left(L_{\text{origin}}, \min_{\substack{j \in \tilde{V} \\ j \neq i \\ x_{j,c}=1}} L(i, j) \right).$$

Then:

$$C_{\text{access}}(x, \tilde{G}) = \sum_{i \in \tilde{V}} \sum_{c \in C} P_{i,c} \left[x_{i,c} L_{\text{local}} + (1 - x_{i,c}) A_{i,c}(x, \tilde{G}) \right],$$

$$C_{\text{rep}}(x, \tilde{G}) = \sum_{i \in \tilde{V}} \sum_{c \in C} \gamma f_c x_{i,c},$$

$$C_{\text{red}}(x, \tilde{G}) = \sum_{c \in C} \sum_{\substack{i,j \in \tilde{V} \\ i<j}} \tilde{w}_{ij} x_{i,c} x_{j,c},$$

$$C_{\text{rel}}(x, x^0) = \sum_{i \in \tilde{V}} \sum_{c \in C} \delta_c |x_{i,c} - x^0_{i,c}|.$$

The total cost under a perturbed graph is

$$J(x, \tilde{G}) = C_{\text{access}}(x, \tilde{G}) + \lambda C_{\text{rep}}(x, \tilde{G}) + \beta C_{\text{red}}(x, \tilde{G}) + \mu C_{\text{rel}}(x, x^0).$$

7.5 Stage I: Greedy Topology-Aware Initialization

The greedy stage builds the cache placement incrementally. At each step, the algorithm selects the node–content pair that gives the largest decrease in expected total cost.

Let x be the current placement. For each feasible pair (i, c), define the marginal gain:

$$\Delta(i, c \mid x) = \mathbb{E}_{\tilde{G} \sim \mathcal{P}(G)} \left[J(x, \tilde{G}) - J(x^{(i,c)}, \tilde{G}) \right],$$

where $x^{(i,c)}$ is the placement obtained by setting $x_{i,c} = 1$, provided the capacity constraint remains satisfied.

At each iteration, the greedy algorithm chooses

$$(i^*, c^*) = \arg\max_{(i,c)} \Delta(i, c \mid x),$$

and updates the placement accordingly. The process stops when all caches are full or when no candidate produces positive marginal gain.

7.6 Stage II: DRL-Based Refinement

The greedy solution is then used as the initial state for DRL refinement. The DRL agent learns a placement-improvement policy under random topology perturbations.

7.6.1 State Representation

At each decision step, the state includes:

- the current perturbed graph $\tilde{G}$,
- the current cache placement x,
- node capacities and free cache slots,
- node-wise demand statistics $P_{i,c}$,

Algorithm 1 Greedy Topology-Aware Cache Initialization

Require: Nominal graph G, perturbation process $\mathcal{P}(G)$, content catalog C, capacities $\{K_i\}$, initial placement x^0

Ensure: Greedy cache placement x

Initialize $x \leftarrow x^0$
Mark all available slots at each node
repeat
  bestGain $\leftarrow 0$, bestPair $\leftarrow \varnothing$
  for each node i **do**
    for each content $c \in C$ such that $x_{i,c} = 0$ and node i has free capacity **do**
      Construct tentative placement $x^{(i,c)}$ by setting $x_{i,c} = 1$
      Estimate marginal gain

$$\Delta(i, c \mid x) = \mathbb{E}_{\tilde{G} \sim \mathcal{P}(G)} \left[J(x, \tilde{G}) - J(x^{(i,c)}, \tilde{G}) \right]$$

      if $\Delta(i, c \mid x) >$ bestGain **then**
        bestGain $\leftarrow \Delta(i, c \mid x)$
        bestPair $\leftarrow (i, c)$
      end if
    end for
  end for
  if bestPair $\neq \varnothing$ **then**
    Update x by caching c at node i for bestPair
  end if
until bestPair $= \varnothing$
return x

- graph-derived features such as degree, weighted degree, or centrality.

7.6.2 Action Space

An action modifies the current placement through one of the following operations:

- add content c to node i,
- remove content c from node i,
- swap content c_1 with c_2 at node i,
- relocate content c from node i to node j.

Each action must preserve feasibility under the cache capacity constraints.

7.6.3 Reward Function

The DRL reward is defined as the negative total cost:

$$r = -J(x, \tilde{G}).$$

Algorithm 2 DRL Refinement under Topology Perturbation

Require: Nominal graph G, perturbation process $\mathcal{P}(G)$, greedy placement x^{greedy}, replay buffer $\mathcal{D}$
Ensure: Refined policy π_θ

```
for episode = 1 to N_episodes do
    Sample perturbed graph G̃ ~ P(G)
    Initialize placement x ← x^greedy
    Construct initial state s_0 = (G̃, x)
    for step t = 0 to T_max do
        Select action a_t ~ π_θ(s_t) using ε-greedy or policy sampling
        Apply action a_t to obtain new feasible placement x'
        Compute reward
                r_t = J(x, G̃) − J(x', G̃)
        Construct next state s_{t+1} = (G̃, x')
        Store transition (s_t, a_t, r_t, s_{t+1}) in replay buffer D
        Update policy parameters θ using mini-batches from D
        Set x ← x'
        if termination condition is satisfied then
            break
        end if
    end for
end for
return learned policy π_θ
```

Equivalently, one may use the improvement in cost between consecutive states:

$$r_t = J(x_t, \tilde{G}_t) - J(x_{t+1}, \tilde{G}_t).$$

This encourages the agent to find placements that reduce access cost, replication overhead, redundancy, and relocation cost simultaneously.

7.6.4 Training Procedure

For each training episode, a perturbed graph $\tilde{G}$ is sampled from $\mathcal{P}(G)$. The episode starts from the greedy placement and the agent interacts with the environment by applying cache modifications until a termination criterion is met (e.g., a maximum number of actions or no further improvement).

7.7 Greedy–DRL Hybrid Inference

At inference time, the final placement is produced in two steps:

1. compute the initial placement using Algorithm 1;
2. refine it using a fixed number of DRL improvement steps from Algorithm 2.

This hybrid strategy preserves the interpretability of the greedy approach while allowing learning-based adaptation to topology perturbations.

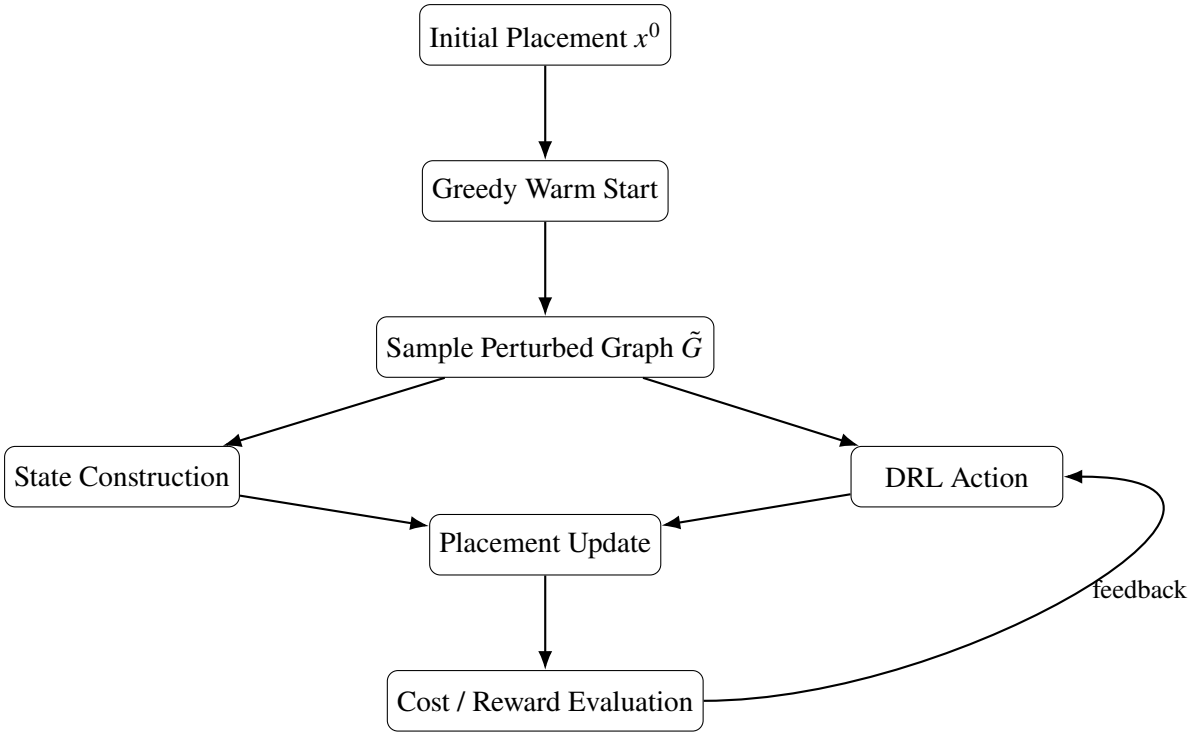

Figure 2: Greedy–DRL hybrid refinement. The greedy stage provides a strong initial placement; the DRL stage improves it under sampled topology perturbations.

7.8 Evaluation Procedure

For each final placement, we evaluate the following:

- expected access cost,
- replication cost,
- redundancy penalty,
- relocation cost,
- total objective value.

These quantities are averaged over multiple perturbed graphs sampled from $\mathcal{P}(G)$. This directly matches the stochastic objective in the problem formulation and measures the robustness of the learned caching strategy.

8 Evaluation

8.1 Experimental Setup

The evaluation is conducted using a graph derived from real-world campus WiFi traces, where nodes represent access points and edges represent user mobility patterns inferred from handoff events. The resulting topology captures realistic connectivity and transition behavior among nodes.

Content request workloads are generated synthetically following a Zipf distribution, which is widely used to model content popularity in network systems. Different skewness parameters are used to simulate

varying levels of content concentration. Cache capacities are uniformly assigned across nodes, and multiple network sizes are considered to evaluate scalability.

Topology perturbations are introduced by randomly adding and removing edges and nodes to simulate dynamic conditions. These perturbations are applied at controlled rates to analyze system robustness under varying degrees of network instability.

8.2 Baselines

The proposed approach is compared against the following baselines:

- **Local Greedy**: Each node independently caches the most popular content without cooperation.
- **Global Greedy**: A centralized greedy algorithm that optimizes placement without considering perturbations.
- **Random Placement**: Content is assigned randomly subject to capacity constraints.

8.3 Metrics

The performance is evaluated using the following metrics:

- **Average Access Cost**: Expected cost of retrieving requested content.
- **Cache Hit Ratio**: Fraction of requests served locally or within the network.
- **Redundancy Ratio**: Degree of duplicate content across nodes.
- **Relocation Overhead**: Cost incurred due to cache updates under perturbations.

8.4 Results

The proposed hybrid approach consistently achieves lower access cost compared to baseline methods, particularly under high perturbation rates. The greedy initialization provides a strong starting point, while the DRL component adapts to topology changes and maintains performance stability.

In contrast, static strategies degrade significantly as perturbation intensity increases. Random placement performs poorly across all scenarios, highlighting the importance of structured optimization.

The results also demonstrate that incorporating relocation cost leads to more stable caching policies, avoiding frequent and unnecessary updates. This confirms the effectiveness of the proposed objective formulation in balancing performance and stability.

8.5 Ablation Study

An ablation study is conducted to evaluate the contribution of each component. Removing the DRL component reduces adaptability, while removing the relocation cost leads to unstable cache behavior. These findings validate the necessity of both components in the overall framework.

9 Discussion

The proposed formulation and methodology provide a structured approach to addressing cooperative caching under dynamic network conditions. Unlike traditional approaches that assume static environments or rely purely on predictive models, our framework explicitly incorporates uncertainty through topology perturbations. This leads to solutions that are inherently more robust and better aligned with real-world deployment scenarios.

A key insight from this work is the trade-off between redundancy and accessibility. While replicating content across multiple nodes improves access latency, excessive redundancy leads to inefficient cache utilization. The inclusion of a redundancy penalty in the objective function ensures that the system balances these competing factors. Similarly, the relocation cost term discourages excessive cache updates, preventing instability in highly dynamic environments.

Another important aspect is the role of topology-awareness. By modeling the network as a graph and incorporating path-based access costs, the caching strategy naturally adapts to the structural properties of the network. Nodes that are central or highly connected tend to store more popular content, while peripheral nodes maintain more localized caches. This emergent behavior aligns with intuitive notions of efficient content distribution.

The hybrid Greedy + DRL approach also reflects a practical design philosophy. Purely learning-based approaches often suffer from slow convergence and high training complexity, especially in large state spaces. On the other hand, purely heuristic methods lack adaptability. By combining both, the system achieves fast initialization and continuous improvement through interaction with the environment.

However, several limitations remain. The reliance on synthetic workload generation may not fully capture complex real-world demand patterns. Additionally, the DRL component introduces computational overhead, which may be challenging for resource-constrained edge devices. Future work can explore lightweight learning methods, federated learning for distributed policy training, and integration with real-time traffic prediction systems.

Overall, the framework provides a solid foundation for studying adaptive edge caching and opens up several directions for further research in cooperative and learning-driven network optimization.

10 Conclusion

This work presents a topology-aware cooperative caching framework designed for dynamic edge networks. By modeling the system as a graph with stochastic perturbations, we capture the inherent variability present in real-world environments. The problem is formulated as a global optimization task that integrates access cost, redundancy, and relocation overhead into a unified objective function.

To solve this problem, we propose a hybrid approach combining greedy initialization with deep reinforcement learning. The greedy component ensures computational efficiency, while the learning component enables adaptive behavior under changing network conditions. This combination results in a robust and scalable caching strategy.

Experimental results demonstrate that the proposed method outperforms traditional caching strategies, particularly in dynamic settings where topology changes frequently. The framework effectively balances

performance and stability, avoiding excessive redundancy and unnecessary cache updates.

Future work will focus on extending the model to incorporate real-time demand prediction, distributed learning mechanisms, and more complex network environments such as multi-tier edge-cloud systems. Additionally, exploring lightweight and interpretable learning models remains an important direction for practical deployment.

Overall, this study provides a principled foundation for adaptive edge caching and highlights the importance of combining structural awareness with learning-based optimization in modern distributed systems.

References

[1] M. Satyanarayanan, "The emergence of edge computing," *IEEE Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[2] M. Chiang and T. Zhang, "Fog and iot: An overview of research opportunities," *IEEE Internet of Things Journal*, vol. 3, no. 6, pp. 854–864, 2016.

[3] L. Bai, J. Cao, M. Zhang, and B. Li, "Collaborative edge intelligence for autonomous vehicles: Opportunities and challenges," *IEEE Network*, vol. 39, no. 2, pp. 52–60, 2025.

[4] K. Shanmugam, N. Golrezaei, A. G. Dimakis, A. F. Molisch, and G. Caire, "Femtocaching: Wireless video content delivery through distributed caching helpers," in *IEEE INFOCOM*, 2013.

[5] N. Golrezaei, A. G. Dimakis, A. F. Molisch, and G. Caire, "Wireless video content delivery through distributed caching and device-to-device communications," *IEEE Journal on Selected Areas in Communications*, vol. 32, no. 6, pp. 1177–1191, 2014.

[6] Y. Dai, D. Xu, Y. Lu, S. Maharjan, and Y. Zhang, "Deep reinforcement learning for edge caching and content delivery in internet of vehicles," pp. 134–139, 2019.

[7] P. Blasco and D. Gündüz, "Learning-based optimization of cache content in a small cell base station," in *Proceedings of the IEEE International Conference on Communications (ICC)*. IEEE, 2014, pp. 1897–1903.

[8] H. Zhou, K. Jiang, S. He, G. Min, and J. Wu, "Distributed deep multi-agent reinforcement learning for cooperative edge caching in internet-of-vehicles," *IEEE Transactions on Wireless Communications*, vol. 22, no. 12, pp. 9595–9609, 2023.

[9] Q. Xu, M. Cheng, J. Jin, Z. Su, D. Fang, Y. Wang, and Y. Wu, "Cooperative edge content caching with popularity prediction in uav-assisted vehicular networks," *IEEE Transactions on Network Science and Engineering*, 2026.

[10] C. Ma, J. Xu, and J. Yang, "Delay-oriented caching strategy for leo satellite-assisted edge caching network," in *2025 IEEE 34th Wireless and Optical Communications Conference (WOCC)*. IEEE, 2025, pp. 22–27.

[11] D. Kotz, T. Henderson, I. Abyzov, and J. Yeo, "Crawdad trace dartmouth/campus(v. 2004-11-09)."

[12] S. Podlipnig and L. Böszörmenyi, "A survey of web cache replacement strategies," *ACM Computing Surveys*, vol. 35, no. 4, pp. 374–398, 2003.

[13] M. A. Maddah-Ali and U. Niesen, "Fundamental limits of caching," *IEEE Transactions on Information Theory*, vol. 60, no. 5, pp. 2856–2867, 2014.

[14] S. Wang, X. Zhang, Y. Zhang, and L. Wang, "A survey on mobile edge networks: Convergence of computing, caching and communications," *IEEE Access*, vol. 5, pp. 6757–6779, 2017.

[15] D. Mardham, S. Madria, J. Milligan, and M. Linderman, "Opportunistic distributed caching for mission-oriented delay-tolerant networks," in *IEEE Wireless On-demand Network Systems and Services (WONS)*, 2018.

[16] R. Cabaniss and S. Madria, "Content distribution in delay-tolerant networks using social context," in *IFIP Wireless and Mobile Networking Conference (WMNC)*, 2014.

[17] F. Wang, S. Emara, I. Kaplan, B. Li, and T. Zeyl, "Multi-agent deep reinforcement learning for cooperative edge caching via hybrid communication,," *IEEE International Conference on Communications*, pp. 1206–1211, 2023.

[18] H. Jiang, J. Guo, Z. Xiao, J. Yang, K. Yang, and G. Min, "Cooperative content caching in vehicular edge computing networks: A two-stage deep reinforcement learning approach," *IEEE Transactions on Mobile Computing*, 2026.

[19] S. Dhara, H. T. Malazi, S. R. Chaudhry, A. Kazmi, and S. Clarke, "Co-operative caching for real-time content retrieval in an integrated space-air-ground network for a post-disaster scenario," *IEEE Transactions on Services Computing*, 2026.

[20] T. Cao, N. Zhang, X. Wang, and J. Huang, "Mobility-aware cooperative caching in vehicular edge computing based on federated distillation and deep reinforcement learning," *IEEE Transactions on Network Science and Engineering*, 2025.

[21] Z. Zhao, L. Zhao, N. Wu, L. Xu, N. Lin, and Z. Tan, "A collaborative caching and offloading approach for vehicular edge computing," *IEEE Transactions on Sustainable Computing*, 2026.

[22] D. Wang, Y. Bai, and B. Song, "A knowledge graph-based reinforcement learning approach for cooperative caching in mec-enabled heterogeneous networks," *Digital Communications and Networks*, 2024.

[23] E. Baştug, M. Bennis, and M. Debbah, "Living on the edge: The role of proactive caching in 5g wireless networks," *IEEE Communications Magazine*, vol. 52, no. 8, pp. 82–89, 2014.

[24] D. Mardham, S. Madria, M. Linderman, and J. Milligan, "Opportunistic distributed caching for mission-oriented delay-tolerant networks," in *IEEE Wireless On-demand Network Systems and Services (WONS)*, 2017.

[25] J. Shuja, K. Bilal, W. Alasmary, H. Sinky, and E. Alanazi, "Applying machine learning techniques for caching in next-generation edge networks: A comprehensive survey," *Journal of Network and Computer Applications*, vol. 181, p. 103005, 2021.

[26] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio, "Unveiling the potential of graph neural networks for network modeling and optimization in sdn," *Proceedings of the 2019 ACM Symposium on SDN Research*, 2019.

[27] Y. Zhao, Y. Zhou, and Z. Han, "Graph neural networks for communication networks: A survey," 2023.